

QuantAnomalies

Project Documentation & Story

A statistical and AI-assisted dashboard that hunts for market anomalies and reframes them as opportunities

1 The Big Idea

Classic finance theory (the “Efficient Market Hypothesis”) makes three tidy assumptions: prices instantly reflect all information, risk is roughly constant, and you cannot predict returns. Decades of evidence show reality is messier — and those mismatches are recurring, studied patterns called **anomalies**.

Our insight: an anomaly is just an **opportunity** seen from the other side. The same crack in the theory that academics flag as “the model is wrong here” is exactly where a careful observer might find an edge. QuantAnomalies makes three of these anomalies visible, measurable, and actionable — then layers an AI analyst on top to explain what it finds.

Textbook says...	Reality (opportunity)	Layman analogy
Prices react instantly	Macro news ripples over weeks (lagged effects)	A stone in a pond — the ripple takes time to reach the edge
Risk is constant	Volatility clusters ; tails are fat	Storms come in clusters, and big ones hit more often than forecast
Returns are unpredictable	Some pairs are cointegrated (tethered)	Two dogs on one leash — they wander but snap back together

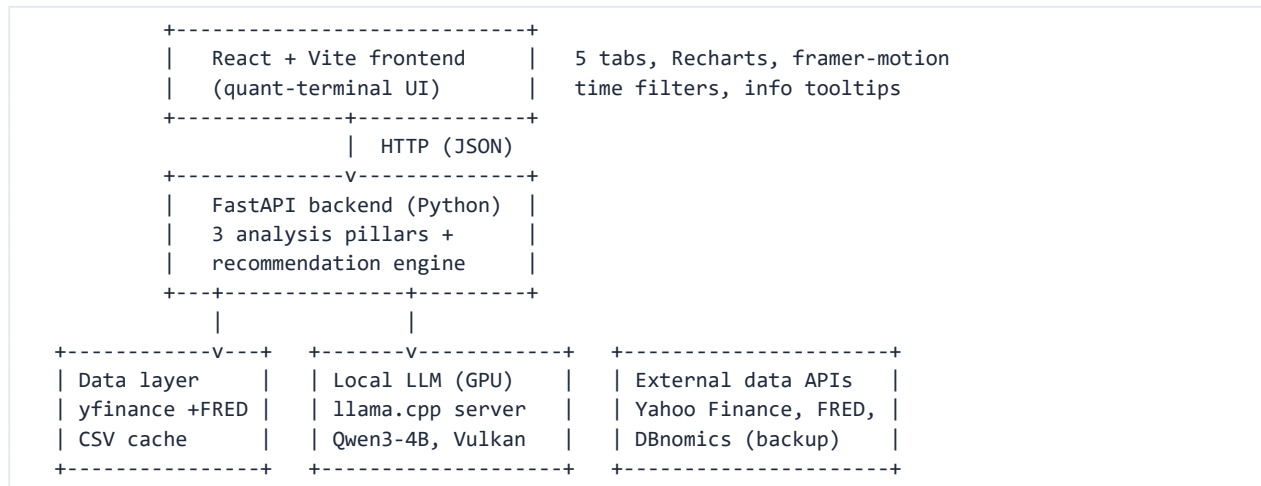
2 What the Project Does

A full-stack web application with five areas:

1. **Overview** — what the project is and how to navigate it.
2. **Opportunities** — the recommendation engine: an automated scan that ranks what’s unusual / actionable right now, with an optional AI analyst note.
3. **Macro Regression** — which macro forces drive an asset, and with what lag.
4. **GARCH Volatility** — how risk changes over time, and how fat the tails are.
5. **Pair Trading** — cointegrated forex pairs and mean-reversion signals.

Everything is **dynamic**: the user can type any ticker (stock, index, crypto, future) or pick any combination of 45 forex pairs, and the analysis regenerates.

3 Architecture



Stack: Python • FastAPI • pandas/numpy • statsmodels • arch • scipy • React 19 • Vite • Recharts • framer-motion • llama.cpp (GPU).

4 The Three Pillars

Pillar 1 — Macro Factor & Lag Regression

Question: Which macro forces move an asset, and how quickly?

Method: Build a monthly dataset of the asset's return plus eight factors (VIX, oil, gold, US dollar, 10-yr yield, Fed Funds, inflation, unemployment). Run OLS regression with **time lags** (0–3 months), Granger-causality tests, and a lagged-correlation heatmap.

Opportunity: If a factor predicts returns weeks ahead, that lag is a window to position before the move. (Sample: S&P 500 $R^2 \approx 0.64$ monthly.)

Pillar 2 — GARCH & Volatility Clustering

Question: How does risk change over time, and are crashes under-estimated?

Method: Fit a **GARCH(1,1)** model to daily returns to estimate day-by-day volatility; test clustering via autocorrelation of squared returns (Ljung-Box); test normality (Jarque-Bera, QQ plot, skew/kurtosis).

Opportunity: A high-volatility regime is a cue to reduce risk; calm regimes can precede expansions. Fat tails warn that standard risk measures understate danger. (Sample: *persistence* ≈ 0.9955 , *excess kurtosis* ≈ 16 — *strongly non-normal*.)

Pillar 3 — Forex Pair Trading

Question: Which currency pairs are tethered, and when is the gap tradeable?

Method: Engle-Granger **cointegration** across all pair combinations; for the best pair, build a hedge-ratio **spread**, standardise it to a **z-score**, and generate buy/sell/exit signals (enter beyond ± 2 , exit near 0); estimate the mean-reversion **half-life**.

Opportunity: Market-neutral "statistical arbitrage" — profit from the gap closing regardless of overall market direction. (Sample: USDCHF/USDJPY cointegrated, $p \approx 0.008$, *half-life* ≈ 64 days.)

5 The Recommendation Engine — "stats detect, AI explains"

This is the layer that turns a dashboard into a recommender. The principle: **deterministic statistics do the detecting; the AI only explains**. The model is handed the detected numbers and is instructed never to invent figures — which is what makes a small local model trustworthy here.

Detectors (v1):

- **Volatility regime** — current GARCH volatility vs its own history.
- **Tail event** — today's move measured in standard deviations.
- **Trend** — price vs 50/200-day averages, near 52-week highs/lows.
- **Forex mean-reversion** — cointegrated pair stretched beyond ± 2 z-score.

Each emits a signal with a 0–1 **severity**; the engine ranks them and computes an overall **confidence**. A rules-based summary always works; the **local LLM** (Qwen3-4B on the GPU) adds a plain-English analyst note in ≈ 4 seconds.

6 Data Pipeline & Robustness

- **Sources:** Yahoo Finance (prices) and FRED (macro), cached as CSV so the app runs offline once data is fetched.
- FRED downloads were failing (HTTP 403) — fixed with a proper request header, plus a **DBnomics** fallback mirror.
- The flaky FRED 10-year series was replaced by Yahoo's $\wedge$ TNX.
- `build_macro_dataset` is **fault-tolerant**: if one factor source fails, that factor is skipped instead of breaking the whole study.
- A data-integrity bug (the VIX and oil caches had been seeded with the wrong series) was detected and corrected; all series now validate to real ranges.

7 The Dashboard (UI)

- A dark “**quant terminal**” theme (deep navy, teal/cyan data, gold highlights), a custom **SVG icon set** (no emojis), and a subtle SVG grid backdrop.
- **Hover info-tooltips** explain every metric in plain English.
- **Time-range filters** (1M–All) on every time-series chart, with the y-axis auto-scaling to the visible window.
- **framer-motion** polish: animated tab transitions, sliding active indicators, and animated confidence/severity bars.

8 The Story So Far

1. **Revived a broken project.** The app wouldn't run — the python command pointed at a dead path; we located the working Anaconda interpreter and got it running again.
2. **Fixed the data.** Repaired FRED access, added fallbacks, swapped the flaky 10-yr source to Yahoo, and cleaned corrupted caches so every value is sane.
3. **Enriched the analysis.** Added gold and the dollar index as macro factors and made the dataset builder fault-tolerant.
4. **Redesigned the interface.** New professional theme, SVG icons, explanatory copy, hover tooltips, and a stealth SVG background.

5. **Built the recommendation engine.** Four anomaly/opportunity detectors plus a provider-agnostic AI client.
6. **Put the AI on the GPU.** Switched the local model from CPU to the RTX 4050 via the bundled Vulkan llama.cpp server — a $\approx 6\times$ speed-up (25s $\rightarrow$ ≈ 4 s).
7. **Added time filters and motion.** Interactive ranges and a proper animation pass across the app.

9 What's Next

Tracked in the roadmap: standardised factor comparisons (so “which factor matters most” is fair), an **options / Black–Scholes** module (implied-vs-model volatility as another opportunity signal), more detectors (macro dislocation, breakout), news-on-chart-click, and a back-test of the signals to turn “opportunities” into measured evidence.

10 How to Run

- **Backend:** `python -m uvicorn main:app --app-dir backend --port 8000` (use the Anaconda interpreter on this machine).
- **GPU AI server (recommended):** `llama-server.exe -m <Qwen3-4B gguf> -ngl 99 -c 4096 --port 8080`
- **Frontend:** `npm run dev` in frontend/ $\rightarrow$ open `http://localhost:5173`.